

savoir où tu es, qui tu es, et si le réseau est OK

```
whoami  
pwd  
hostname  
ip a  
ip r  
ls -lah
```

3) Utilisateurs & groupes (scénarios examen)

Scénario A — Créer un utilisateur + lui donner des droits admin

```
sudo adduser kiki  
sudo usermod -aG sudo kiki  
id kiki  
groups kiki
```

Scénario B — Créer un groupe + ajouter un utilisateur

```
sudo groupadd rh  
sudo usermod -aG rh kiki  
getent group rh
```

4) Fichiers / dossiers / permissions (LE plus important)

Scénario C — Créer /data/kiki avec permissions strictes

```
sudo mkdir -p /data/kiki  
sudo chown kiki:kiki /data/kiki  
sudo chmod 750 /data/kiki  
ls -ld /data/kiki
```

- ~~750~~ 750 = propriétaire : rwx | groupe : r-x | autres : ---
- ~~Principe~~ Principe du moindre privilège : seuls les utilisateurs autorisés peuvent accéder.

5) Installer des paquets (apt)

Scénario D — Installer Apache (serveur web)

```
sudo apt update  
sudo apt upgrade -y  
sudo apt install apache2 -y  
  
sudo systemctl status apache2  
sudo systemctl enable --now apache2  
  
curl -I http://localhost
```

6) Réseau : tests (toujours dans cet ordre)

Scénario E — Pas d'accès internet / pas de ping

```
ip a  
ip r
```

```
ping -c 4 192.168.X.254      # passerelle
ping -c 4 8.8.8.8           # internet (IP)
ping -c 4 google.com        # DNS
nslookup google.com
```

7) Services : vérifier / démarrer / redémarrer

```
sudo systemctl status apache2
sudo systemctl start apache2
sudo systemctl restart apache2
sudo systemctl enable apache2
```

8) Logs : dépannage (le jury adore)

Apache : page blanche / erreur 403/404/500

```
sudo tail -n 50 /var/log/apache2/error.log
sudo tail -n 50 /var/log/apache2/access.log
```

Logs système (général)

```
sudo journalctl -xe
```

9) Ports : vérifier si un service écoute

```
sudo ss -tulpen
sudo ss -tulpen | grep -E '(:80|:22|:443)'
```

10) Pare-feu (UFW) — si demandé sécurité

```
sudo apt install ufw -y
sudo ufw enable
sudo ufw status verbose
```

```
sudo ufw allow ssh
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

11) Cron : automatisation

```
crontab -e
```

```
# Exemple : toutes les 5 minutes
*/5 * * * * echo "$(date) test" >> /home/kiki/cron.txt
```

```
crontab -l
tail -n 20 /home/kiki/cron.txt
```

```
sudo systemctl status cron
```

12) Scénario EXAM complet : test du port HTTP (serveur + client)

Sur le serveur Debian

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable --now apache2
ip a
curl -I http://localhost
```

Sur le client Debian

```
ping -c 4 IP_SERVEUR
curl -I http://IP_SERVEUR
```

13) Preuves (captures indispensables)

- ~~€€€€€€€~~ ip a + ip r (serveur)
- ~~€€€€€€€~~ systemctl status apache2 (active)
- ~~€€€€€€€~~ curl -I http://localhost (code 200)
- ~~€€€€€€€~~ curl -I depuis le client vers IP_SERVEUR
- ~~€€€€€€€~~ ls -ld /data/kiki (droits)
- ~~€€€€€€€~~ id kiki / groups kiki (user + groupes)
- ~~€€€€€€€~~ crontab -l + tail fichier cron.txt (automatisation)

14) Phrases d'oral (prêtes à dire)

- ~~€€€€€€€~~ « Je commence par vérifier mon contexte : utilisateur, chemin courant, configuration IP et route par défaut. »
- ~~€€€€€€€~~ « J'installe et j'active le service, puis je valide localement (curl localhost) avant de tester depuis un client. »
- ~~€€€€€€€~~ « En cas d'échec, je consulte les logs (journalctl / logs Apache) pour identifier précisément l'erreur. »
- ~~€€€€€€€~~ « Je sécurise l'accès avec des permissions strictes et/ou un pare-feu, conformément au moindre privilège. »